Shotput Box Packing Algorithm and Efficiency Report*

Stephanie Hutson

November 2021

ORIGINALLY PUBLISHED ON MEDIUM JULY 7, 2016 AS A INDUSTRIAL WHITE PAPER FOR SHOTPUT INC. [1]

Abstract

The Shotput box packing algorithm is used for determining the best box size for an order we receive into our system. The algorithm was developed in order to minimize shipping costs, and reduce complexity for the warehouse and operations teams. Through a series of tests comparing the effectiveness of the Shotput box packing algorithm against a similar algorithm, pyShipping, it was determined Shotput packs significantly and consistently more space efficiently than pyShipping.

*For orginal publications see https://medium.com/the-chain/efficiency-of-the-shotput-packing-algorithm-a690e914d49c and https://medium.com/the-chain/solving-the-box-selection-algorithm-8695df087a4

[1] Shotput Inc. is now defunct as a company, any reference to present technologies may be outdated. As a non-academic paper, citations will be found in footnotes.

1 Introduction

Shotput Inc. is a third party fulfillment company based in Oakland, California serving small businesses whose inventory is too small for traditional services. Orders for shipments are uploaded by an individual company or directly issued to the Shotput database through use of an in house API. Once an order is received into the system, the operations and warehouse team must prepare shipments for order and ship them to the customer. A key question however is which box or boxes are necessary for the shipment.

1.1 Motivation

For many fulfillment companies, box selection is done poorly[2]. Many larger companies have found it more cost effective to select a box that is too large than to have to figure out which one would work best. As a consumer, it may be comical or mildly annoying to receive a box far too large for its contents; as a business, using a box that is too big is hugely inefficient in cost, materials, and energy consumption.

Practically speaking, for a small business, using too large a box or packing inefficiently results to significant money loss. To prevent movement and damage to the contents of a package, you'll often use additional packing material. Not only that, but carrier rates are often affected by volume; a one inch difference in a single dimension may cost up to $5[3].

There is an environmental impact to using too large a box as well. The obvious impact is the result of using more materials, less obvious is the energy consumption from the carrier itself. If your box is 25% larger than a workable alternative, that means you could only ship 4 in an equal amount of space for every 5 that you would if your box were the right size, ultimately requiring more vehicles to deliver an equal number of packages.

We found that by using a packing algorithm, we could quote more precise shipping costs and

[2] https://consumerist.com/2015/07/10/heres-why-amazons-stupid-shipping-gang-wrapped-some-bubble-wrap-in-brown-paper/

[3] For USPS, the difference in shipping cost between an 18x18x15" parcel and an 18x18x16" parcel is $5, while UPS charges approximately $0.35 for every additional inch in the dimensions of a parcel. Source: https://postcalc.usps.com/ and https://www.theupsstore.com/tools/estimate-shipping-cost

reduce the number of decisions that need to be made by the operations team, thus allowing them to focus their energies on higher priority issues.

Prior to writing the Shotput Box Packing Algorithm, the company had been using pyShipping[4], a library developed for the same purpose, and the only open source 3D bin packing library available[5]. Although sophisticated, the operations team found that the suggested parcel organization was not accurate[6] enough for their needs, so I was enlisted to develop something better.

1.2 Design Decisions

There are certain requirements and limitations that needed to be considered when developing the algorithm, for instance each client has a specific set of sizes of box available to them for use, and each box has weight limitations. To keep the program modular, we decided to do an initial selection from the set of available boxes that would fit the item[7] with the longest dimensions, then after running the algorithm select if necessary a smaller box for the boxes with the smallest items in them.

Because pyShipping was already in place when I developed the new algorithm, I opted to maintain a similar data structure for the Shotput algorithm as pyShipping to allow the team to easily switch to the new algorithm without additional overhead.

1.3 The Algorithm

An optimal box-packing algorithm is an algorithmically hard problem. Both brute-force branching, and more sophisticated optimal algorithms rapidly become infeasible for orders with more than a couple items in them.

Because Shotput is a small startup with 8 employees and therefore resources, both fiscal and human, are limited, the algorithm also needed to be developed and tested over a short enough period of time to be useful. Not only that, but the output needs to be simple enough that the

[4]https://github.com/hudora/pyShipping

[5]Martello, Silvano & Pisinger, David & Vigo, Daniele & Boef, Edgar & Korst, Jan. (2007). Algorithm 864: General and robot-packable variants of the three-dimensional bin packing problem.. ACM Trans. Math. Softw.. 33. 7.

[6]"Accurate" in this paper this means how tightly packed the items in a box are. For instance the expectation is that the packing algorithm should digitally pack boxes as space efficiently as a human on the warehouse floor would.

[7]An "item" is an individual product needing to be packed and shipped

warehouse staff can easily replicate, and the run-time of the algorithm must be short enough that a human use does not notice a lag. The complete details are proprietary[8], to meet these constraints, we use a heuristic algorithm that seeks to replicate some common-sense choices a human would make.

From the literature, we use the strategy of First Fit Descending[9]: doing an initial sort of the items by longest dimension to pack the largest items first. Following that, the algorithm places an item in a box with its longest edge along the shortest edge of the box that it will fit. For instance in two dimensions, if you have a 7x10" square box and a 7x5" item to be "packed" the 7" edge of the item is packed along the 7" edge of the box, leaving behind a remaining 7x5" empty space that can hold another item, which, due to use of first fit descending, we know to be at least as small as the last item packed.

After inserting the algorithm into the Shotput codebase, we found that the algorithm met the needs of the operations and warehouse teams, performing better than pyShipping, which itself performs better than more theoretically complicated algorithms.

2 Testing and Performance

Although anecdotally, the new packing algorithm was performing well for the operations team, we decided to rigorously test the accuracy and time component of both the Shotput Box Packing Algorithm and pyShipping. Because pyShipping measured its success against Pisinger's `3bpp` algorithm[10], and find that pyShipping is slower, but more effective in packing, we may use those results to add context to our own.

[8]This is no longer true. For the complete algorithm see:
https://github.com/stephaniemhutson/BoxPackingAPI/blob/master/packing_algorithm.py

[9]Dósa, György, and Jirí Sgall. "First Fit bin packing: A tight analysis." 30th International Symposium on Theoretical Aspects of Computer Science (STACS 2013). Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2013.

[10]Boef, Edgar & Korst, Jan & Martello, Silvano & Pisinger, David & Vigo, Daniele. (2005). Erratum to "The Three-Dimensional Bin Packing Problem": Robot-Packable and Orthogonal Variants of Packing Problems. Operations Research. 53. 735-736. 10.1287/opre.1050.0210.

	Box Size	Item Size	Items Packed
Small	100-200	40-100	10-30
Medium	100-200	20-100	20-100
Large	50-400	1-50	100-1000

Table 1: Parameters for the small medium and large test cases. All sizes are in arbitrary "units", and the key factor is the ratio between box size and item size.

2.1 Methodology

To rigorously test the accuracy and run-time of each algorithm against each other, we simplified the performance metric to limit the number of box sizes available in each trial to one.

For this sub-problem we measure for both algorithms:

- The number of boxes used to pack all items
- The number of items in the last packed box[11]
- The run-time

We generated three random data sets, small, medium and large, each with 10,000 samples. The "large" data set uses a proportionally large range of item sizes and a large number of items, while small uses a small range of sizes and a small number of items. Parameters for each of the tests cases found in Table 1. In real terms, the small and medium data sets are most representative of our current use case. To test the run-time, we used the "medium" proportions for item and box size and calculated the run time packing different numbers of items. Because the run-time of the Shotput algorithm in our use case does not create a human perceivable lag, run-time is the least important of the test cases, however it is important to consider it for future scalability constraints.

2.2 Results

We ran 10,000 samples for each test case, the results of which can be found in Tables 2 and 3 and Figure 1, . In case of a tie, we looked into whether both algorithms placed all items in one box. If they did, they were excluded from Figure 1. Run-time results can be found in Table 4.

[11]This is used as an imperfect proxy for the smallest usable box size, which pyShipping does not calculate.

	Shotput	**Tie**	**pyShipping**
Small			
Fewest Parcels Required	8588	1354	58
Fewest Items[12]	633	455	266
Medium			
Fewest Parcels Required	9780	219	1
Fewest Items	194	21	44
Large			
Fewest Parcels Required	7793	2189	18
Fewest Items	423	1766	0

Table 2: Results from test cases Small, Medium, and Large.

	Mean	**Standard Deviation**	**Median**
Small			
Num Fewer Parcels	2.50	2.01	2
% Fewer Parcels	29.5%	16.3%	33.3%
Medium			
Num Fewer Parcels	5.91	4.16	5
% Fewer Parcels	41.1%	11.2%	42.8%
Large			
Num Fewer Parcels	3.68	5.57	2
% Fewer Parcels	45.1%	16.9%	50%

Table 3: Statistics about the results. For "fewer" we are using *parcels used for pyShipping minus parcels used for Shotput.*

Average Run-time (ms)		
Num Items Packed	Shotput	pyShipping
10	0.3	0.07
50	4.5	0.8
100	15.0	2.6
200	63.5	10.4
300	140.0	23
400	221.1	35.9
500	307.4	49.2

Table 4: Run-time results

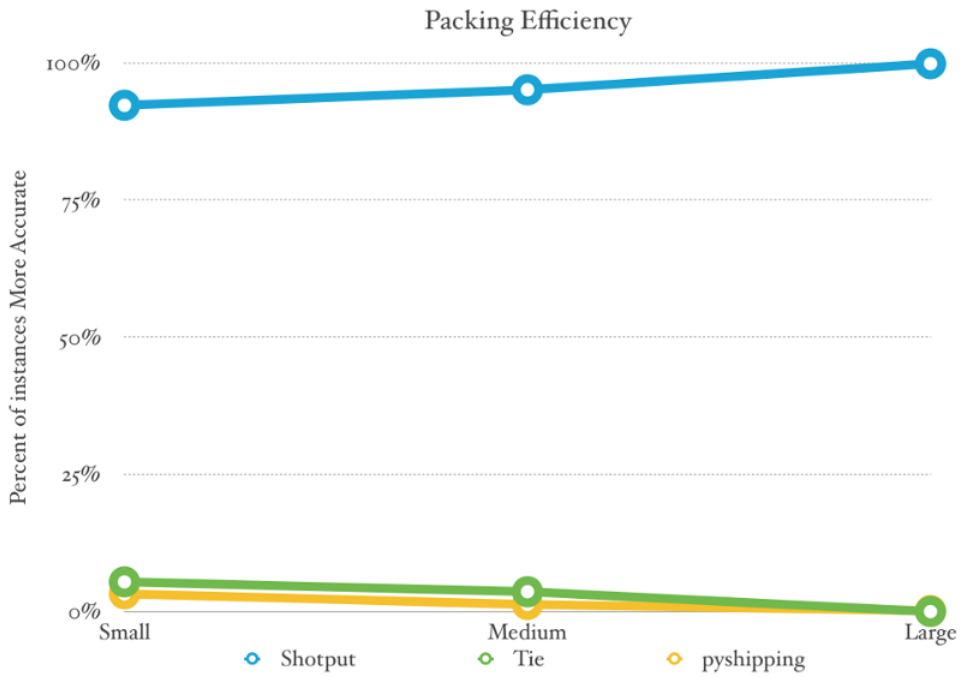

Figure 1: The success rate of each algorithm in each of the test cases.

We see from tables 2 and 3 that the Shotput algorithm outperforms the pyShipping algorithm the vast majority of the time. The Shotput algorithm uses between 29.5% and 45.1% fewer parcels on average to pack the same items. All of these values are statistically significant.

We also see that the run-time of Shotput scales much more quickly than pyShipping. Translated into a power function we have

$$y = 0.0043x^{1.8} \text{ and } y = 0.0012x^{1.7}$$

respectively.

3 Conclusions

The results were very clear. After compiling the data from 28,230 trials over test cases Small, Medium and Large, (30,000 trials minus ties where all items were packed in one bin for both algorithms), Shotput's box packing algorithm successfully required fewer parcels for the same items 92.5% of the time and packed more efficiently 97.0% of the time. We can say with certainty that the Shotput shipping algorithm is more precise and more accurate than pyShipping. This supports our anecdotal evidence.

In our current phase of development, and the current use cases (represented by test cases 2 and 3) for the Shotput box packing algorithm, the algorithm is fast enough, and from a user standpoint does not preform badly relative to pyShipping. Both algorithms have a Big O better than $O(n)^2$, which is far better than a brute force approach. Because Shotput scales at a fractionally higher exponential rate and a noticeably greater linear rate, as complexity increases, as does the difference in run-times. This data can be used to entice the next stage in optimization and development.